Vues, contrôles et modificateurs en SwiftUI

420-249-AH - Programmation mobile iOS

1. SwiftUI : une interface composée de vues

En SwiftUI, presque tout ce qui apparaît à l'écran est une :

```
View
```

Exemples :

- Texte
- Image
- Bouton
- Champ de saisie
- Interrupteur
- Liste
- Conteneur

Une interface est donc construite en assemblant plusieurs vues.

2. Exemple simple

```swift
struct ContentView: View {
    var body: some View {
        Text("Bonjour SwiftUI")
    }
}
```

Ici, Text est une vue.

3. Combiner plusieurs vues

Pour afficher plusieurs éléments, on utilise généralement des conteneurs :

```
VStack
HStack
ZStack
```

Exemple :

```
VStack {
    Text("Bienvenue")
    Image(systemName: "iphone")
    Button("Continuer") {
        print("Bouton appuyé")
    }
}
```

4. Text

Text permet d'afficher du texte.

```
Text("Bienvenue")
```

On peut appliquer plusieurs modificateurs :

```
Text("Bienvenue")
    .font(.title)
    .bold()
```

5. Styles de police

Quelques styles courants :

```
Text("Titre")
    .font(.title)

Text("Sous-titre")
    .font(.headline)

Text("Texte normal")
    .font(.body)

Text("Information secondaire")
    .font(.caption)
```

6. Image

Pour afficher une image provenant des ressources :

```
Image("profil")
```

Pour utiliser une icône du système :

```
Image(systemName: "person.circle")
```

Les icônes système utilisent SF Symbols.

7. Modifier une image système

```
Image(systemName: "battery.75percent")
    .font(.system(size: 40))
```

On peut également utiliser :

```
.resizable()
.scaledToFit()
```

pour certaines images.

8. Exemple avec une image

```
Image("profil")
    .resizable()
    .scaledToFit()
    .frame(width: 120, height: 120)
```

9. Button

Un bouton contient généralement un titre et une action.

Exemple :

```
Button("Connexion") {
    print("Connexion...")
}
```

Le bloc :

```
{
    print("Connexion...")
}
```

est une closure.

10. Bouton personnalisé

```
Button {
    print("Profil sélectionné")
} label: {
    Label(
        "Profil",
        systemImage: "person"
    )
}
```

11. Label

Label permet de combiner facilement un texte et une icône.

Exemple :

```
Label(
    "Paramètres",
    systemImage: "gear"
)
```

12. TextField

TextField permet de saisir du texte.

```
@State private var nom = ""
```

Puis :

```
TextField(
    "Nom",
    text: $nom
)
```

Le symbole $ permet ici de fournir une liaison vers la valeur.

Nous approfondirons @State et les bindings dans une présentation dédiée.

13. SecureField

Pour une saisie confidentielle :

```
@State private var motDePasse = ""
```

```
SecureField(
    "Mot de passe",
    text: $motDePasse
)
```

Le contenu n'est pas affiché en clair.

14. Toggle

Toggle représente un interrupteur.

```
@State private var notifications = true
```

Puis :

```
Toggle(
    "Notifications",
    isOn: $notifications
)
```

15. Exemple de petit écran

```
struct ProfilView: View {
    @State private var nom = ""
    @State private var notifications = true

    var body: some View {
        VStack(spacing: 20) {
            Image(systemName: "person.circle")
                .font(.system(size: 60))
            Text("Profil")
                .font(.title)
            TextField(
                "Nom",
                text: $nom
            )
            Toggle(
                "Notifications",
                isOn: $notifications
            )
            Button("Enregistrer") {
                print(nom)
            }
        }
        .padding()
    }
}
```

16. Qu'est-ce qu'un modificateur ?

Un modificateur permet de modifier l'apparence ou le comportement d'une vue.

Syntaxe :

```
vue
    .modificateur()
```

Exemple :

```
Text("Bonjour")
    .font(.title)
    .bold()
    .padding()
```

17. Chaînage des modificateurs

Les modificateurs peuvent être enchaînés :

```
Text("Connexion")
    .font(.headline)
    .padding()
    .frame(maxWidth: .infinity)
```

Chaque modificateur retourne une nouvelle vue.

18. padding

Ajoute de l'espace autour d'une vue.

```
Text("Bonjour")
    .padding()
```

Ou :

```
.padding(20)
```

Ou encore :

```
.padding(.horizontal)
```

19. frame

Permet de définir ou influencer la taille d'une vue.

```
Text("Bienvenue")
    .frame(
        width: 200,
        height: 50
    )
```

Très fréquent :

```
.frame(maxWidth: .infinity)
```

pour occuper l'espace disponible.

20. background

Permet d'ajouter un arrière-plan.

```
Text("Important")
    .padding()
    .background(.gray.opacity(0.2))
```

21. cornerRadius

Exemple :

```
Text("Message")
    .padding()
    .background(.gray.opacity(0.2))
    .cornerRadius(10)
```

On obtient un bloc avec coins arrondis.

22. foregroundStyle

Permet de définir le style visuel du contenu.

```
Text("Erreur")
    .foregroundStyle(.red)
```

ou :

```
Image(systemName: "checkmark.circle")
    .foregroundStyle(.green)
```

23. opacity

```
Text("Désactivé")
    .opacity(0.5)
```

La valeur varie généralement entre 0 et 1.

24. disabled

Pour désactiver une interaction :

```
Button("Continuer") {
}
.disabled(true)
```

Exemple dynamique :

```
.disabled(nom.isEmpty)
```

Le bouton devient inaccessible si le nom est vide.

25. hidden

```
Text("Secret")
    .hidden()
```

La vue n'est plus visible, mais elle peut continuer à occuper sa place dans la mise en page.

26. L'ordre des modificateurs est important

Considérons :

```
Text("SwiftUI")
    .padding()
    .background(.gray)
```

Ici : padding puis background. L'arrière-plan couvre également l'espace ajouté.

Mais :

```
Text("SwiftUI")
    .background(.gray)
    .padding()
```

produit un résultat différent.

> **Avec SwiftUI :** l'ordre des modificateurs peut changer le résultat visuel.

27. Réutiliser un style

Si plusieurs vues utilisent les mêmes modificateurs :

```
Text("Accueil")
    .font(.headline)
    .padding()

Text("Profil")
    .font(.headline)
    .padding()
```

on peut ensuite organiser ou factoriser ces styles.

Nous verrons plus tard comment créer des composants et modificateurs réutilisables.

8. Exemple - Carte de profil

```
struct CarteProfil: View {
    var body: some View {
        VStack(spacing: 12) {
            Image(systemName: "person.circle.fill")
                .font(.system(size: 70))
            Text("Alex Martin")
                .font(.title2)
                .bold()
            Text("Développeur iOS")
                .foregroundStyle(.secondary)
            Button("Voir le profil") {
                print("Profil")
            }
            .buttonStyle(.borderedProminent)
        }
        .padding()
        .frame(maxWidth: .infinity)
        .background(.gray.opacity(0.1))
        .cornerRadius(16)
        .padding()
    }
}
```

29. Quelques styles de bouton

SwiftUI fournit des styles prêts à l'emploi.

```
.buttonStyle(.bordered)
.buttonStyle(.borderedProminent)
```

Exemple :

```
Button("Enregistrer") {
}
.buttonStyle(.borderedProminent)
```

30. Principe important en SwiftUI

On évite généralement de penser : créer une vue, puis modifier directement cette vue.

On pense plutôt : décrire l'interface souhaitée à partir de l'état actuel.

Exemple :

```
if estConnecte {
    Text("Connecté")
} else {
    Text("Déconnecté")
}
```

L'interface dépend directement de l'état.

31. Exercice — Écran de profil

Créer une vue :

```
ProfilView
```

contenant :

- Une icône de profil
- Un titre
- Un champ Nom
- Un champ Courriel
- Un Toggle pour les notifications
- Un bouton Enregistrer

Éléments requis :

Utiliser au minimum :

```
Text
Image
TextField
Toggle
Button
VStack
```

et les modificateurs :

```
.font()
.padding()
.frame()
.background()
.cornerRadius()
.foregroundStyle()
```

Comportement :

Le bouton Enregistrer doit afficher dans la console :

```
Console
Nom : ...
Courriel : ...
Notifications : true/false
```

32. Exercice — Validation du formulaire

Désactiver le bouton si :

- nom est vide
- OU courriel est vide

avec :

```
.disabled(...)
```